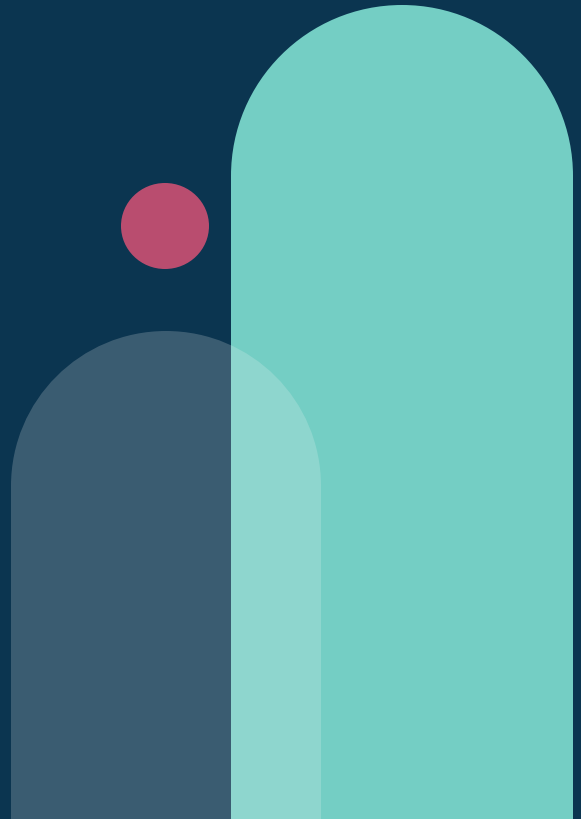
LangChain Fundamentals



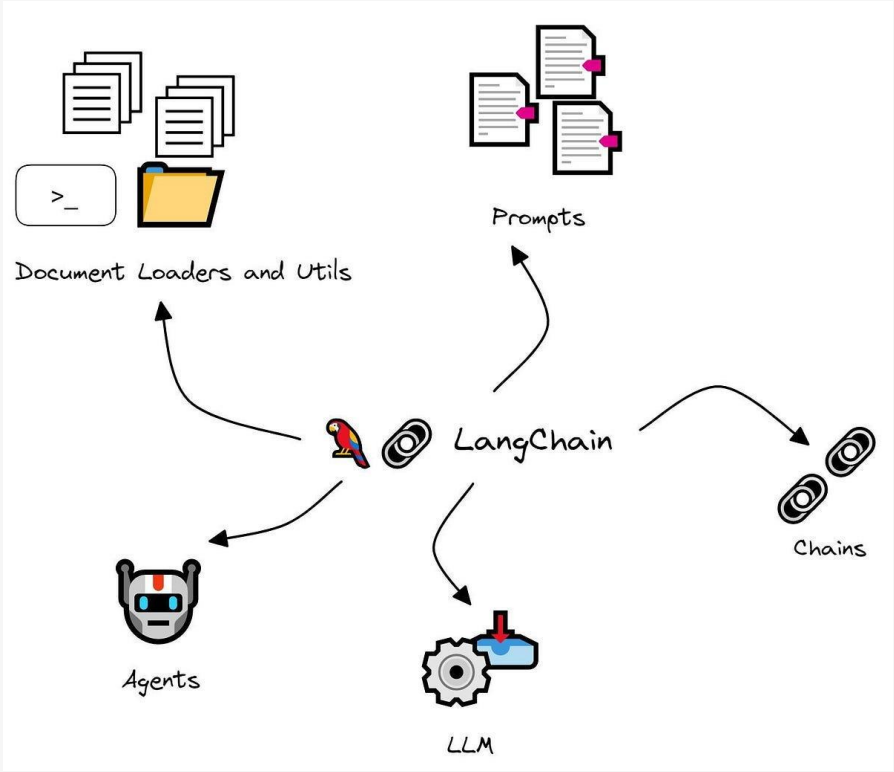
01

LangChain



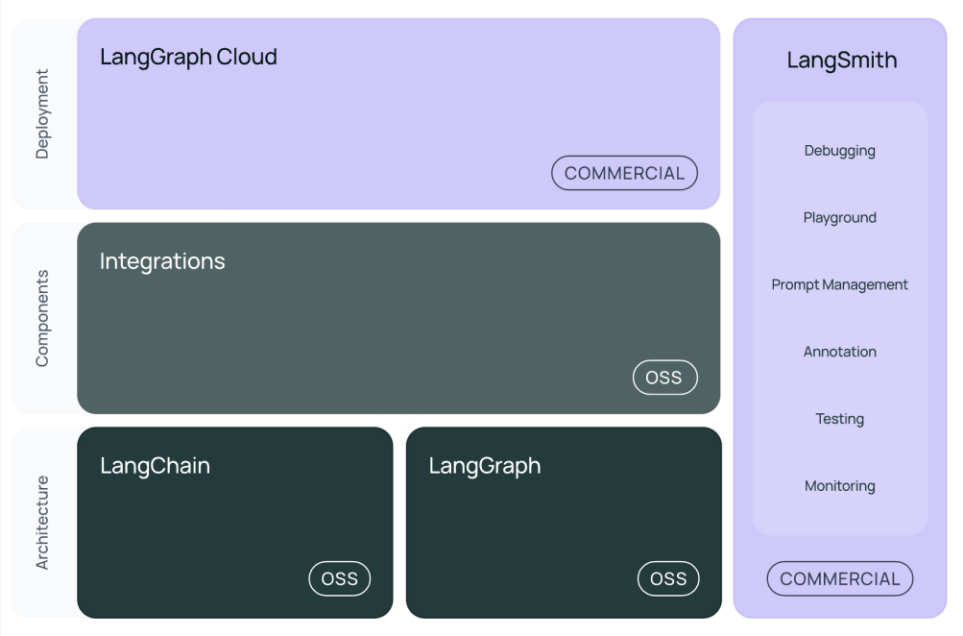
What is LangChain

- **LangChain** is a framework for developing applications powered by large language models (LLMs).
- Integrates open-source libraries and tools to build, debug, and deploy LLM applications efficiently.



LangChain Framework

- **LangChain-Core:** Base abstractions and interfaces.
- **LangGraph:** For building stateful, multi-actor applications.
- **LangServe:** Deploy LangChain chains as REST APIs.
- **LangSmith:** Debug, test, and monitor LLM applications.



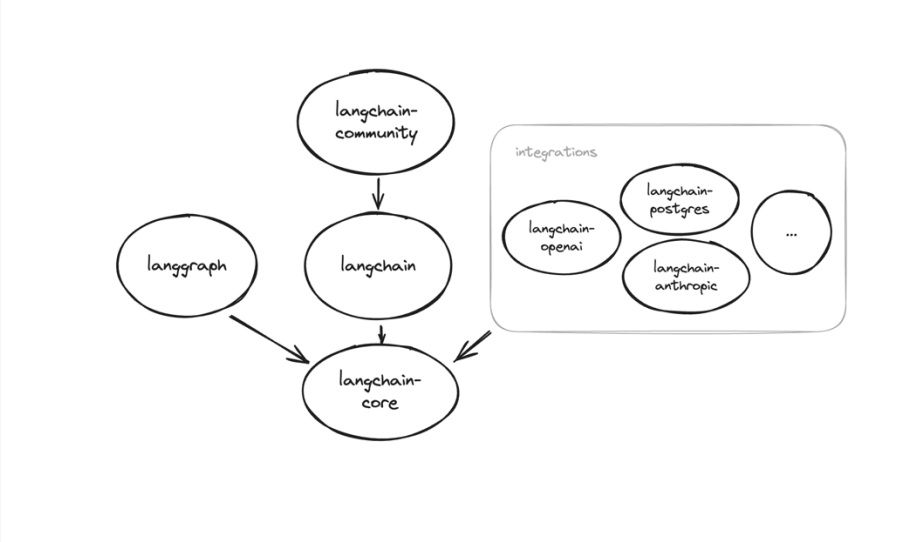
LangChain Architecture

LangChain Architecture

- **LangChain-Core:** Core components for building LLM applications.
- **Partner Packages:** Integration with popular third-party LLMs (e.g., OpenAI, Anthropic).
- **LangChain-Community:** Community-driven integrations for various components (LLMs, vector stores, retrievers).

Architecture Highlights

- Separation of core and integration packages.
- Flexibility to plug in various LLM providers and components.
- Lightweight and modular design for scalable applications.

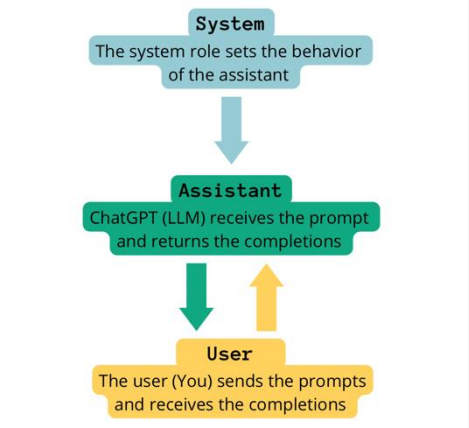


Basic Components

ChatModels

Language models that use a sequence of messages as inputs and return chat messages as outputs.

Supports roles like user, assistant, system, and tool for better interaction modeling.



LLMs (Language Models)

Models that take a string as input and return a string as output.

Can also handle messages similar to ChatModels but with a simpler interface.

Messages

Fundamental units of communication in LangChain.

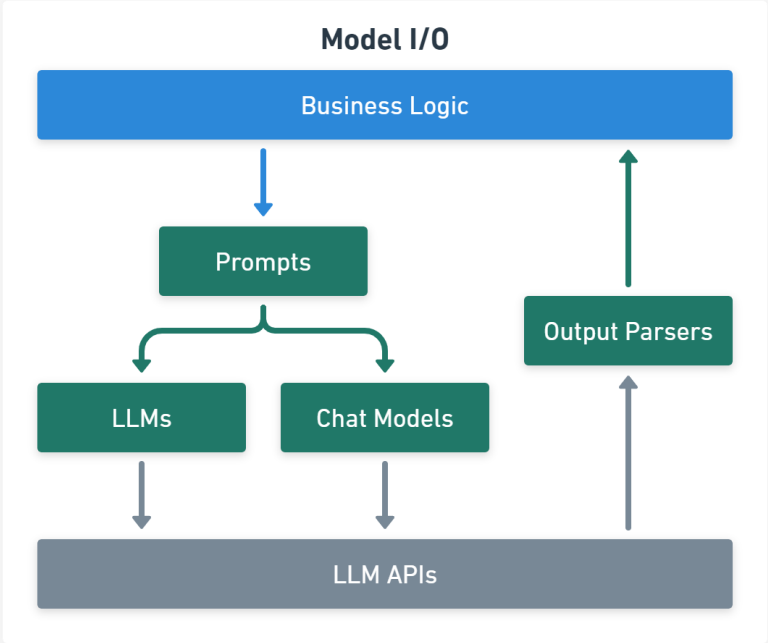
Types include:

- **HumanMessage:** Represents messages from the user.
- **AIMessage:** Represents messages generated by the model.
- **SystemMessage:** Provides context or instructions to the model.

Basic Components

PromptTemplates
Templates that guide the model's behavior by structuring input prompts.
Supports dynamic input with variables, e.g., "Translate the following into {language}: {text}".

OutputParsers
Tools for transforming model outputs into structured formats.
Essential for tasks like extracting specific data or formatting output for further processing.



LCEL

Introduction to LCEL
(LangChain Expression Language)

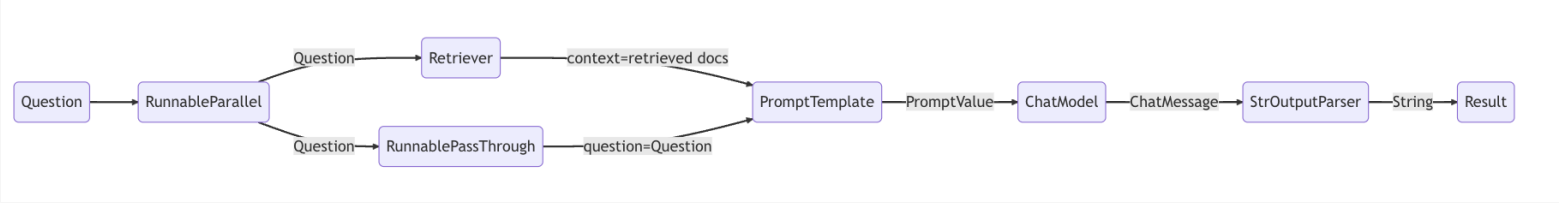
A powerful way to combine LangChain components using the pipe (|) operator.

```
chain = prompt | model | output_parser
```

Benefits of Using LCEL

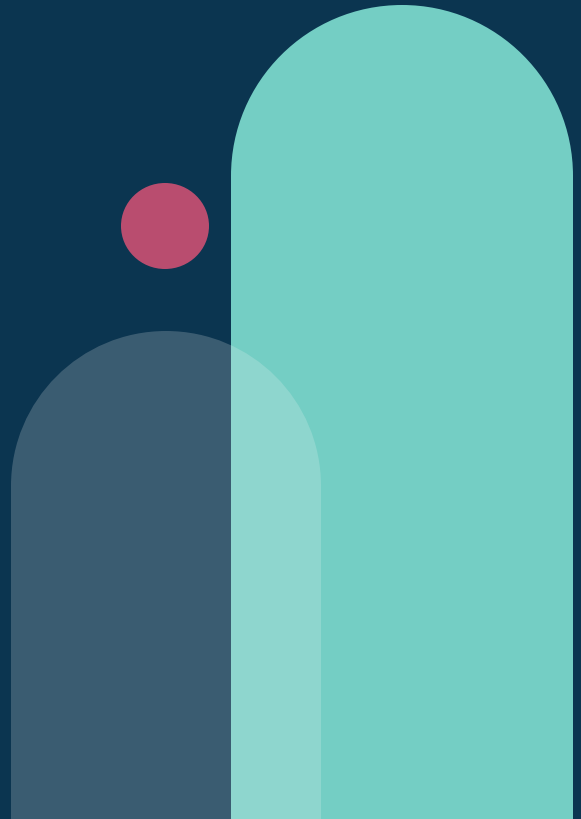
- Optimized streaming and async support.
- Enhanced debugging with traceability through LangSmith.
- Easy chaining of components, even in complex applications.

```
setup_and_retrieval = RunnableParallel(
    {"context": retriever, "question": RunnablePassthrough()}
)
chain = setup_and_retrieval | prompt | model | output_parser
```



02

HandsOn



Thanks!

Do you have any questions?

